为什么 MCU 工程不写 malloc：静态内存池

嵌入式 Linux 手记　2026-11-02　公众号文章 · 原链接 mp.weixin.qq.com

先猜理由。这段代码在 Linux 上是标准写法，搬到 STM32 工程里，为什么会被老工程师直接打回?

```
void handle_request(void) {
    char *buf = malloc(512);   /* 动态分配 */
    if (buf == NULL)
        return;
    process(buf);
    free(buf);
}
```

判空有了、用完就放，看起来无懈可击。但在 MCU 世界它踩了三条线：分配时间不确定、内存碎片不可预测、失败路径没法穷尽测试。这一篇讲清楚"为什么禁"与"用什么替"：MISRA 的红线画在哪、FreeRTOS 官方五套分配器各管什么场景、定长对象池怎么做到 O(1)，以及功能安全认证里的终极答案——链接期就锁死全部内存。篇 20 讲过外部碎片是长期运行的慢性病，这一篇是那本账的嵌入式续篇。

一、MISRA 的两条红线

MISRA C:2012 是汽车、医疗、工控等功能安全领域的通行编码标准，它对动态内存分配的立场写在两条规则里。Directive 4.12：不得使用动态内存分配；Rule 21.3 更具体：禁用 `<stdlib.h>` 的 `malloc` 、 `calloc` 、 `realloc` 、 `free` 四件套。标准级的"明令禁止"，不留商量余地。

背后是四条理由，每一条都指向嵌入式系统的命门。其一，分配时间不确定： `malloc` 在空闲链上找货，找到哪块、要不要合并、要不要切分，耗时随仓库状态波动——实时系统的每一步都要有上界，"通常很快"在实时性审查里等于不合格。其二，碎片不可预测：篇 20 的外部碎片是随机游走，设备跑三个月后的堆长什么样，没人能画出来，长跑必挂。其三，失败路径难以穷尽测试：每次 `malloc` 都多一个 `NULL` 分支，测试要覆盖"第 N 次分配恰好失败"的所有组合，组合数爆炸。其四，认证审计困难：功能安全认证要求每行代码的行为可证明，一个行为依赖运行历史的分配器，审计员没法签字。



图：MISRA C:2012 两条红线与四条理由。时间、碎片、测试、审计，条条指向嵌入式系统的命门。

二、FreeRTOS 五套分配器：按需选一套

FreeRTOS 官方给出五套分配器源码（`heap_1.c` 到 `heap_5.c`），按需选一套编译进工程：注意是选一套替换，不是让四件套进场。五套的差异与适用一张表说清：

方案	策略	适用场景
heap_1	只分配、不实现 free	对象创建后终身使用，确定性最强
heap_2	支持释放，但不合并相邻空闲块	分配/释放尺寸固定的场景
heap_3	薄封装标准 malloc/free（挂起调度器保线程安全）	快速移植存量代码
heap_4	首次适应 + 相邻空闲块合并	通用默认选择，抗碎片
heap_5	heap_4 + 多内存区支持	RAM 物理不连续的芯片

选型逻辑顺着确定性往下排：`heap_1` 最极端，没有 `free` 就没有碎片、没有释放竞争，代价是内存只出不进，只适合对象"出生即终身"的系统（任务、队列、信号量这类创建后不销毁的资源）。`heap_2` 补上了释放，但不做合并：同尺寸块即还即用是它的舒适区，尺寸一杂就开始积累碎片。`heap_4` 是大多数项目的答案：首次适应加合并，篇 20 讲的 coalesce 在这里就是抗碎片的日常手段。`heap_5` 只解决一件事：芯片有两片物理上不连续的 RAM 时，把多段内存注册成同一个堆。`heap_3` 最没脾气，只是给标准 `malloc/free` 挂上调度器锁，用于存量代码快速移植，功能安全场景别选它。



图：五套分配器选型表与两条彻底路线。按确定性排序选一套编译进工程。

三、定长对象池：O(1) 的确定性

五套之外还有更彻底的思路：定长对象池。每个池只管一种尺寸：池子初始化时把 N 个等大的对象用链表串起来，分配是摘链头，释放是插回链头，两个操作都是 O(1)、耗时恒定、与历史无关。

代价是内部碎片：配额 64 字节的对象哪怕只用 10 字节，剩下的也是池子的（篇 20 的内部碎片在此从缺点变成设计取舍）。收益是外部碎片彻底消失：块与块之间永远同构，不存在"总量够、拼不出"的问题。这套思路在工业界到处都是：lwIP 网络协议栈的内存池、Linux 内核的 slab 分

配器（对象缓存）都是同一个骨架，只是复杂度不同。嵌入式项目手写一个几十行的版本就够用：一个数组当存储区、一个空闲链表头、初始化时串起来，完事。

四、彻底静态：链接期锁死全部内存

功能安全认证场景的终极答案是零运行期分配：FreeRTOS 提供 `xTaskCreateStatic()` 一族 Static API，任务控制块（TCB）、栈、队列全部由调用方声明为全局变量传入，内核不再自己找内存。每个任务的栈直接写进链接脚本或全局数组，链接器在链接期就把全部内存摆上桌面——MAP 文件里每个字节的去向一目了然。

这就是"静态内存池"的完整含义：不是换一种更安全的动态分配，而是把"分配"这个动作整体从运行期挪到链接期。运行期少一个不确定因素，测试矩阵就少一个维度，认证审计就少一叠表格。代价是灵活性：想要新对象得改代码重新编译，而嵌入式设备的固件生命周期以年计，编译期的死板恰恰是运行期的可靠。

写在最后

1. MISRA C:2012 两条红线：Directive 4.12 禁动态分配、Rule 21.3 点名 malloc/calloc/realloc/free；四条理由：时间不确定、碎片不可预测、失败路径穷尽不了、认证审计签不了字。
2. FreeRTOS 五套分配器按确定性排序：heap_1 无 free 最强、heap_2 同尺寸即还即用、heap_4 首次适应+合并是通用默认、heap_5 管不连续 RAM、heap_3 只为移植存量代码。
3. 定长对象池用内部碎片换外部碎片归零：分配释放 O(1) 恒定耗时，lwIP 与内核 slab 同思路，几十行代码可自写。
4. 彻底静态是功能安全的事实标准：`xTaskCreateStatic()` 把内存全部搬到链接期，MAP 文件即内存全景图。
5. 桌面思维与 MCU 思维的分水岭：前者优化"平均情况"，后者证明"最坏情况"。

静态内存池管住了"分配"，还有一个更隐蔽的坑在后面：程序申请 1GB 内存、实际只碰了 1MB，Linux 不但不报错还答应得爽快。下一篇《程序申请 1G 内存为什么没崩：overcommit 与 OOM》，看内核的过度承诺与最后的杀手。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。